

Minesweeper — System Architecture Document v2.0

Oct 2nd, 2026 · create by Cyn, edit by Nathan, Aidan, Maxwell, Warren, Dillion, Fabri

Team 22 · EECS 581 Fall 2026 · Project 2: Maintenance & Extension

1. Purpose and Scope

This document describes how the Group 22 Minesweeper works, map documentations for easy search, and describes how data moves. The system is a static browser game: plain HTML, CSS and JavaScript, no build step, no server, no modules. It is opened by double-clicking index.html. Three scripts share one global scope and load in a fixed order (grid.js, then ai.js, then [ui.js](#)).

This version (v2.0) supersedes the inherited Project 1 architecture document (v1.1), which covered only the single-player game. It adds:

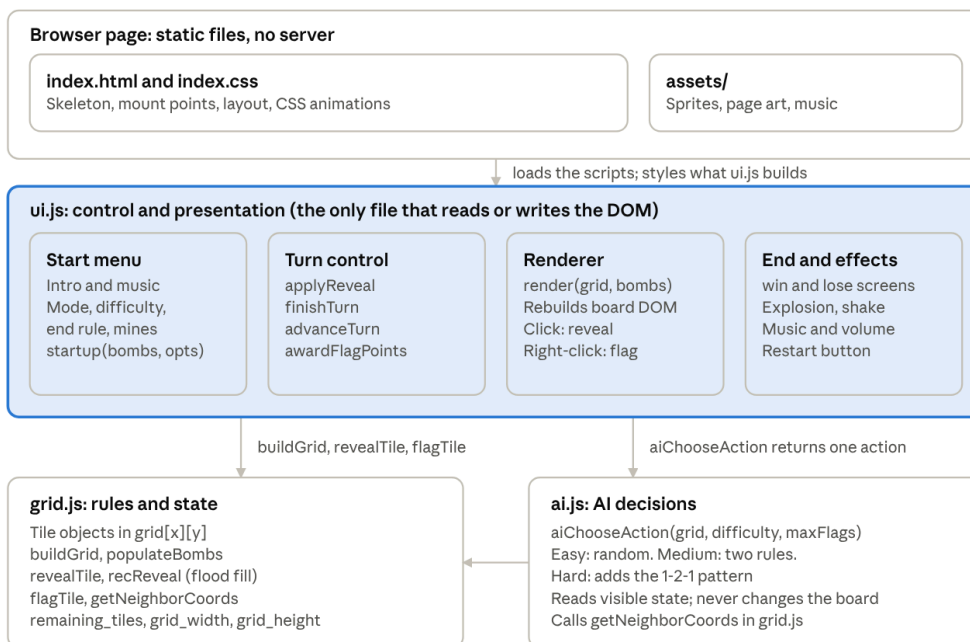
- the AI solver (ai.js) with three difficulty levels;
- local 2-player and Player-vs-AI modes with alternating turns;
- two end-of-game rules: instant death and points;
- the guaranteed zero-valued first click;
- sprites, sound, screen shake and the scoreboard;
- the reorganised repository layout (assets/, docs/).

Sections 2 to 5 cover architecture, components, data structures and data flow. Sections 6 to 8 cover module interfaces, game rules and presentation. Sections 9 and 10 cover how to extend the code and what is still limited.

2. System Overview

The shaded box is the control layer. *ui.js* calls *grid.js* for rules and *ai.js* for AI moves, then redraws the page; *ai.js* calls only the neighbour helper in *grid.js*. Nothing below *ui.js* knows the page exists.

ui.js drives the game; grid.js and ai.js never touch the page



system architecture · 3 scripts, 1 control layer

3. Components

The code splits into a *rules layer (grid.js)*, a *decision layer (ai.js)* and a presentation and *control layer (ui.js)*.

Component	Layer	Responsibility	Key functions
grid.js	Rules and state	Builds the board, places mines, computes neighbour counts, reveals tiles (flood fill), toggles flags, reports game status	buildGrid, populateBombs, setTileNeighboringBombCounts, getNeighborCoords, checkTile, revealTile, flagTile, recReveal
ai.js	Decision	Chooses one move per AI turn from what a human could see; never mutates the board	aiChooseAction, aiRandomAction, aiCollectHiddenUnflagged, aiFindBasicDeductions, aiFind121Deductions
ui.js	Control and presentation	Start menu, rendering, click and right-click handling, turn order, scoring, win and loss screens, audio, animations	startup, render, applyReveal, finishTurn, advanceTurn, takeAiAction, runAutoSolve, guaranteeZeroFirstClick, awardFlagPoints, win, lose
index.html	Page skeleton	Loads the three scripts and stylesheet; provides the mount points #intro-screen, #game-area, #container-two, #main-container, #turn-indicator	none
index.css	Styling	Layout, sprite-sheet animations (title fire, explosion), screen shake and fall-off, per-player background colour, scoreboard cards, menu panel	keyframes fall, explode, fire-animation, screen-shake, fall-off
assets/images, assets/audio	Static media	Sprites, page art, icons, theme music	referenced by path from ui.js, index.css, index.html
docs/	Documentation	Architecture, testing, sprint backlog, meeting logs for both teams	none

grid.js: rules layer

- buildGrid(height, width, numBombs) creates a width by height array of Tile objects, places mines, fills in neighbour counts, and sets remaining_tiles to the number of safe tiles.
- revealTile(grid, x, y) is the only entry point for revealing. It ignores flipped or flagged tiles, returns "Game Over: Loss" on a mine, starts recReveal on a zero tile, and returns "Victory" when remaining_tiles reaches 0. Otherwise it returns "Playing".
- recReveal flips a zero tile, then visits all eight neighbours, recursing on zeros and flipping numbered tiles without recursing. Flagged and already flipped neighbours are skipped.
- flagTile(grid, x, y, owner) toggles a flag and records which player placed it in flagOwner.
- getNeighborCoords(x, y) returns in-bounds neighbour coordinates. It was added in Project 2 for the AI.

ai.js: decision layer

- aiChooseAction(grid, difficulty, maxFlags) returns one action or null.

- Easy picks a random hidden, unflagged tile. Medium applies two rules to each revealed number: if the hidden neighbours equal the remaining mine count they are all mines (flag), and if the flagged neighbours already equal it the other hidden neighbours are safe (reveal). Hard adds the 1-2-1 pattern, tried only when the basic rules find nothing.
- When no rule applies, the AI falls back to a random reveal. Flag actions are offered only while the flags placed are fewer than maxFlags.

ui.js: control and presentation layer

- startup(bombs, options) builds the grid with buildGrid(10, 10, bombs), resets mode state, and calls render.
- render(grid, bombs, first_run) rebuilds the board DOM from the grid on every action and attaches the click and right-click handlers.
- finishTurn is the single place where a result from grid.js becomes a consequence: win, loss, next turn, or continue.
- The window load handler runs the intro screen, starts the music, and builds the start menu.

4. Key Data Structures

The game state is one two-dimensional array of Tile objects plus a handful of global variables.

One instance per square, defined in grid.js.

Field	Type	Meaning
isFlipped	boolean	Tile has been revealed
isFlagged	boolean	A flag is currently on the tile
flagOwner	null, 1 or 2	Player who placed the flag; null when unflagged or in solo mode
isBomb	boolean	Tile hides a mine
numSurroundingBombs	integer 0 to 8	Mines among the eight neighbours; null until counts are computed

The board

grid[x][y] where x is the column (0 to grid_width - 1) and y is the row (0 to grid_height - 1). The outer array has length width, each inner array has length height. The current game is 10 by 10, so the two are equal and the order cannot be seen in play. A future non-square board must keep this order.

Globals in grid.js

Name	Meaning
grid_width, grid_height	Board size, set by buildGrid; used for bounds checks by recReveal, getNeighborCoords and ai.js
remaining_tiles	Safe tiles not yet revealed; the win test is remaining_tiles == 0

Globals in ui.js

Name	Values	Meaning
gameMode	"solo", "multiplayer", "vsai"	Selected mode
endMode	"death", "points"	End-of-game rule
aiDifficulty	"easy", "medium", "hard"	AI strength

Name	Values	Meaning
aiAutoSolve	boolean	AI plays every turn itself
activePlayer	1 or 2	Whose turn it is; in vs-AI mode player 2 is the AI
boardLocked	boolean	Ignore clicks during the AI turn or auto-solve
scores	{1: n, 2: n}	Points per player (points mode)
firstRevealPending	boolean	True until the first reveal of the game has been made
pendingExplosion	[x, y] or null	Mine just uncovered, waiting for the explosion animation
autoSolveTimer	interval id or null	Handle for the auto-solve loop
currentMusic	Audio	Track currently playing
BOMB_PENALTY	3	Points lost for uncovering a mine
WRONG_FLAG_PENALTY	2	Points lost for a flag on a safe tile
PLAYER_COLORS, FLAG_SPRITES	maps keyed by 1 and 2	Colour name and flag image per player

Messages between modules

Structure	Shape	Produced by	Consumed by
Game status	string "Playing", "Victory" or "Game Over: Loss"	revealTile	finishTurn, win_loss_continue
Start options	{ mode, difficulty, autoSolve, endMode }	start menu in the load handler	startup
AI action	{ type: "flag" or "reveal", x, y }, or null	aiChooseAction	takeAiAction, performAiAction
Coordinate pair	[x, y]	getNeighborCoords, aiCollectHiddenUnflagged	ai.js rules

5. Data Flow

One turn always follows the same path: an action enters from a click or from the AI, is applied to the grid, and is judged in finishTurn, which either redraws the board and passes the turn or ends the game.

Every human click and AI move passes through finishTurn



data flow of one turn · 2 entry points, 3 outcomes

Things the picture leaves out:

- **Flags.** A flag action calls flagTile and skips revealTile. In multiplayer and vs-AI it then calls finishTurn with "Playing", so a flag uses up the turn. In solo mode it only re-renders.
- **Points mode.** applyReveal catches a mine before revealTile: the tile is flipped, the active player loses 3 points, and the result is "Playing", so the game continues.
- **Locking.** While the AI is thinking, boardLocked is true and the click handler returns at once, so a human cannot act out of turn.
- **Rendering.** render is not part of the rules path. It reads the grid, never changes it, and is called after every action.

Where each piece of data lives

Data	Created	Changed by	Read by
grid[x][y] (Tile objects)	buildGrid	revealTile, recReveal, flagTile, guaranteeZeroFirstClick	render, ai.js, finishTurn, win, lose
remaining_tiles	buildGrid	revealTile, recReveal	revealTile
activePlayer, boardLocked	startup	advanceTurn	click handlers, updateTurnIndicator
scores	startup	applyReveal, awardFlagPoints	winSummary, buildScoreCard

Data	Created	Changed by	Read by
pendingExplosion	applyReveal	playPendingExplosion	playPendingExplosion

6. Module Interfaces

Dependencies run one way: ui.js calls grid.js and ai.js; ai.js calls grid.js; grid.js calls nothing else. There are no modules, every function and global below is visible everywhere, so these contracts are conventions, not enforced by the language.

Caller	Function	Inputs	Output	Side effects
ui.js	buildGrid(height, width, numBombs)	integers	grid[x][y] of Tile	sets grid_width, grid_height, remaining_tiles
ui.js	revealTile(grid, x, y)	board, coordinates	"Playing", "Victory" or "Game Over: Loss"	flips tiles; lowers remaining_tiles
ui.js	flagTile(grid, x, y, owner)	board, coordinates, player 1 or 2 or null	none	toggles isFlagged and flagOwner; ignores flipped tiles
ui.js	setTileNeighboringBombCounts(grid)	board	board	recomputes every numSurroundingBombs; called after mines are moved by the first click
ai.js	getNeighborCoords(x, y)	coordinates	array of [x, y]	none; reads grid_width, grid_height
ui.js	aiChooseAction(grid, difficulty, maxFlags)	board, "easy", "medium" or "hard", total mine count	{type, x, y} or null	none; the caller applies the action

Rules a change must respect:

- All reveals go through revealTile (or applyReveal in ui.js, which wraps it). Setting isFlipped directly skips the remaining_tiles bookkeeping and breaks the win check.
- ai.js may read isFlipped, isFlagged, numSurroundingBombs and the isBomb of tiles that are already flipped. It must not read isBomb of hidden tiles, or the AI would cheat.
- aiChooseAction returns exactly one action. ui.js decides whether and how to perform it, so turn order and scoring stay in one place.
- Anything that changes the board must be followed by render(grid, bombs, false); the page never updates itself.
- Script order in index.html matters only for definitions used at load time. All cross-file calls happen after load, but keep grid.js first so its globals exist before ui.js runs.

7. Game Modes, Turns, Scoring and States

The start menu produces an options object. gameMode decides who acts; endMode decides how the game ends. The two are independent. One action is either one reveal or one flag placed or removed. A player cannot change an opponent's flag, and the number of flags on the board cannot exceed the mine count.

Mode	gameMode	Who acts	Turn rule
Solo	"solo"	One human	No turns; every action re-renders the board
2 Player	"multiplayer"	Player 1 (red), Player 2 (blue)	One action per turn, then advanceTurn swaps activePlayer
Vs AI	"vsai"	Player 1 human, Player 2 is the AI	After the human acts, the board locks and takeAiAction runs 700 ms later
Vs AI, auto-solve	"vsai" with aiAutoSolve	AI only	runAutoSolve performs one AI action every 500 ms; no turn swap

End-of-game rules

Rule	endMode	Mine uncovered	Round ends when	Winner
Instant death	"death"	Game over; the player who uncovered it loses	Mine hit, or every tile is flipped or flagged	Clearing the board makes both sides winners
Points	"points"	Tile flips, that player loses BOMB_PENALTY (3) points, play continues	Every tile is flipped or flagged	Highest score after awardFlagPoints

awardFlagPoints runs once at the end: each flag gives its owner +1 if on a mine and -WRONG_FLAG_PENALTY (2) if on a safe tile. In solo mode a mine always ends the game, and victory is remaining_tiles == 0.

Game states

revealTile reports the status as a string, and finishTurn turns it into an outcome. In 2-player and vs-AI games finishTurn overrides "Playing" with "Victory" as soon as every tile is flipped or flagged, so the round can end before all safe tiles are open.

Status string	Meaning	Next step in finishTurn
"Playing"	Game continues	render, play any pending explosion, advanceTurn
"Victory"	Round complete	win(grid): summary or scores, restart button
"Game Over: Loss"	A mine ended the game	lose(grid): reveal mines, fade-out sequence, loss message, restart button

Restarting reloads the page (window.location.reload), so no state needs to be reset by hand.

First click

The first reveal of every game calls guaranteeZeroFirstClick. It empties the clicked tile and its neighbours of mines, moves each mine to the first free tile elsewhere, then recomputes all neighbour counts. The first reveal is therefore always a zero tile and opens a cascade.

8. Audio, Animation and Assets

Presentation is driven by ui.js and index.css and has no effect on game rules. Removing any item here leaves the game playable.

Feature	Where	How it works
Intro screen	#intro-screen, load handler	A black overlay with "CLICK TO BEGIN". The click is needed because browsers block audio until the user interacts
Title sequence	.title, .title-fire, .title-explosion, .title-shadow in index.css	CSS keyframe animations (fall, explode, fire-animation, shadow-grow) using sprite sheets; body.game-started triggers them
Music	load handler	minesweeper_theme.mp3 plays once on the first click; when it ends, minesweeper_default.mp3 plays on a loop. currentMusic always points at the active track
Volume button	#volume-button	Toggles currentMusic between play and pause and swaps volume_on.png and volume_off.png
Tile sprites	spriteImg, render	render gives each button a class (hidden-tile, revealed-tile, has-sprite, and tile-zero to tile-three for the numbers 0 to 3; higher numbers use the default style), writes the number as text, and inserts flag or bomb images
Explosion and shake	playPendingExplosion, shakeScreen	When pendingExplosion is set, the explosion sprite sheet plays on that tile and .screen-shake is added to #game-area
Loss sequence	lose, dropEverythingOffScreen	Pauses music, reveals mines, then fades to black. Solo games also run the fall-off animation after 900 ms
Turn colours	updateTurnIndicator	Adds turn-p1 or turn-p2 to <body>; CSS fades the background to red or blue
Scoreboard	#turn-indicator, buildScoreCard	One card per player with flag counter and score. It sits outside #game-area so the shake does not move it

Assets

Folder	Files	Used by
assets/images	bomb, mine_bomb, flag_red, flag_blue, explode-sprite-sheet, ongoing_fire_sprite_sheet, minesweeper_page_art, volume_on, volume_off	ui.js (paths in strings), index.css (url(...)), index.html
assets/audio	minesweeper_theme.mp3, minesweeper_default.mp3	ui.js

Asset paths are plain strings in three files. After moving or renaming a file, search all of ui.js, index.css and index.html for its old name.

9. How to Extend the System

Each common change touches a small, known set of places. Follow the row for your change, then re-run the manual test cases in docs/.

To add	Edit	Steps
A new AI rule	ai.js	Write aiFindXDeductions(grid, mines, safes) that pushes [x, y] pairs into mines (to flag) and safes (to reveal). Call it from aiChooseAction for the difficulty that should use it. Read only visible state
A new AI difficulty	ai.js, ui.js	Add a branch in aiChooseAction; add the option to the difficulty dropdown in the load handler
A new game mode	ui.js	Add a value for gameMode, a button in the start menu, and handling in advanceTurn (who acts next) and finishTurn (when the round ends)
A new end rule or score	ui.js	Extend endMode, adjust applyReveal (what a mine does) and awardFlagPoints (end-of-round scoring); add constants beside BOMB_PENALTY
A different board size	ui.js, grid.js	Replace the fixed buildGrid(10, 10, bombs) in startup with chosen values; extend numtoLetter beyond J; fix the width and height naming in populateBombs and setTileNeighboringBombCounts (section 10); update the mine range check
A new tile appearance	index.css, ui.js	Add a CSS class and set it in the class chain inside render
A new sound or image	assets/, ui.js	Add the file, reference it by a path string, and check section 8 for where paths are used
A new rule in the rules layer	grid.js	Keep it free of DOM code; return a value and let ui.js react to it

Guidelines that keep the structure intact:

- Keeping grid.js and ai.js free of DOM access makes them testable from the browser console.
- Add new status strings to revealTile only together with a matching branch in win_loss_continue, or the new state is silently treated as "Playing".
- Do not call render inside grid.js or ai.js; ui.js re-renders after each action.
- Do not read hidden isBomb values in [ai.js](#).

10. Known Limitations and Changes from v1.1

These are properties of the code on main as reviewed for this document. None blocks normal play; each is a place a future team should expect surprises.

Area	Limitation	Where	Effect
Board size	Fixed 10 by 10	startup calls buildGrid(10, 10, bombs)	Other sizes need the changes listed in section 9
Row labels	numtoLetter handles only A to J	ui.js	Boards taller than 10 rows would lose labels
Naming	populateBombs and setTileNeighboringBombCounts read width from grid[0].length and height from grid.length, the reverse of the grid[x][y] layout	grid.js	Harmless on a square board; wrong on a rectangular one

Area	Limitation	Where	Effect
Mine input	Checked only as a number between 10 and 20; decimals are not rejected	start menu in ui.js	A value such as 10.5 reaches startup
Mine placement	populateBombs retries random cells until it finds a free one	grid.js	Would loop forever if mines exceeded tiles; not reachable with the 10 to 20 limit
Status codes	Game status is a free string, compared in several places	revealTile, finishTurn, win_loss_continue	A typo creates a silent bug; constants would be safer
Rendering	render rebuilds the whole board on every action	ui.js	Fine at 100 tiles; slow for large boards
Globals	All state is global and shared across files	all	Load order and name clashes matter; no modules
AI input	aiChooseAction is handed the full grid, including hidden mines	ai.js	The AI honours a read-only-visible-state convention, but nothing enforces it
Debug code	printGrid is left in grid.js	grid.js	Unused in play
Page markup	The tab icon link uses rel="minesweeper tab icon", which browsers ignore	index.html	No tab icon; should be rel="icon"
Restart	Implemented as a page reload	addRestartButton	Resets everything, including audio

Changes from v1.1

Topic	v1.1 (Project 1)	v2.0 (this document)
Scripts	grid.js, ui.js	adds ai.js; load order grid.js, ai.js, ui.js
Modes	Solo only	Solo, 2 Player, Vs AI, auto-solve
End rules	Mine ends the game	Instant death or points
Flags	Plain toggle	Per-player owner (flagOwner), colour sprites, limit equal to mine count
First click	Could hit a mine	Always a zero tile (guaranteeZeroFirstClick)
Helpers in grid.js	none for neighbours	getNeighborCoords
Repository	Media and logs at the root	assets/images, assets/audio, docs/ with logs for both teams
Presentation	Basic grid	Sprites, explosion, shake, fall-off, scoreboard, per-player background